ĐẠI HỌC QUỐC GIA TP HCM

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**



# BÁO CÁO CUỐI KỲ

Môn: **Hệ CSDL tiên tiến**

Thực hiện **Nhóm 1**:


PHẠM THỊ THÚY NGỌC – 250104046

PHẠM THÚY VI – 250104058

CUNG PHƯƠNG ANH - 250104029

PHẠM THỊ THU VÂN - 250104057


Giảng viên**: PGS.TS NGUYỄN ĐÌNH THUÂN**

TP HỒ CHÍ MINH – NĂM 2026

**Bảng phân công công việc**

| Họ và tên | MSSV | Bảng phân công |
|---|---|---|
| Phạm Thị Thuý Ngọc | 250104046 | Nội dung chính (Đặt vấn đề & 4 thách thức) |
| Phạm Thuý Vi | 250104058 | Chạy thực nghiệm bài báo |
| Cung Phương Anh | 250104029 | Hướng phát triển từ bài báo |
| Phạm Thị Thu Vân | 250104057 | Phân tích tính SOTA cùa bài báo |

1. Nội dung chính ..... 3
1.1. Vấn đề bài báo giải quyết ..... 3
1.2. Các thách thức ở 4 giai đoạn của pipeline học máy ..... 4
1.3. Kết quả và điều nhóm rút ra ..... 4
2. Tính SOTA của bài báo ..... 5
3. Phát triển từ bài báo (What, How, Why) ..... 5
3.1. WHAT – Bài toán nhóm đề xuất ..... 5
3.2. HOW – Cách thực hiện ..... 6
3.3. WHY – Tại sao làm như vậy ..... 7
4. Cài đặt minh họa và kế hoạch thực nghiệm ..... 7
4.1. Môi trường ..... 7
4.2. Thí nghiệm 1: Kiểm chứng kết luận của bài báo ..... 8
4.3. Thí nghiệm 2: Đánh giá hướng phát triển của nhóm ..... 8
4.4. Hạn chế ..... 9
Tài liệu tham khảo ..... 9

# 1. Nội dung chính

## 1.1. Vấn đề bài báo giải quyết

Với mỗi câu SQL, bộ tối ưu truy vấn phải chọn thứ tự nối bảng, thuật toán nối và cách quét từng bảng. Số phương án tăng rất nhanh theo số bảng: trường hợp xấu nhất lên tới O(n!), và bài toán thuộc lớp NP-hard [1, §1]. PostgreSQL chọn kế hoạch bằng quy hoạch động cộng với mô hình chi phí. Nhưng mô hình chi phí dựa vào ước lượng số dòng (cardinality), mà ước lượng này thường sai, nên kế hoạch được đánh giá là tối ưu chi phí vẫn có thể chạy chậm.

Bộ tối ưu dùng học máy (LQO) khắc phục điểm này bằng cách học từ thời gian chạy thật. Mức độ can thiệp khác nhau: Neo [2] và Balsa [4] tự xây toàn bộ kế hoạch; LEON [5] xếp hạng các kế hoạch ứng viên; Bao [3] chỉ chọn bộ gợi ý (hint set) rồi để PostgreSQL tự lập ra kế hoạch; HybridQO (mô hình lai) [6] dùng chi phí để sinh ứng viên và dùng mô hình dự đoán độ trễ để chọn. Các bài báo của từng phương pháp đều báo cáo LQO nhanh hơn PostgreSQL, nhưng mỗi bài dùng phần cứng, cấu hình và tập dữ liệu khác nhau, vì vậy khó để so trực tiếp với nhau [1, §7.1, §8.1.4].

Lehmann và cộng sự [1] đặt ra vấn đề: khi đưa mọi phương pháp về cùng điều kiện và tính đủ mọi khoản thời gian, LQO có còn thắng PostgreSQL không? Bài báo không đề xuất LQO mới. Tác giả coi LQO là một bài toán học máy gồm bốn giai đoạn: sinh dữ liệu → mã hóa → huấn luyện → đánh giá [1, Hình 1]. Theo họ, các nghiên cứu truóc tập trung vào mô hình.

Bài báo có ba đóng góp:

- Phân tích thách thức ở từng giai đoạn kèm khuyến nghị [1, §3–§6]
- Xây dựng khung đánh giá đầu-cuối giúp kết quả so sánh được và tái lập được [1, §7]
- dùng khung này đánh giá lại năm LQO trên hai bộ benchmark JOB và STACK [1, §8]

## 1.2. Các thách thức ở 4 giai đoạn của pipeline học máy

Mỗi giai đoạn trả lời một câu hỏi: dữ liệu có đáng tin không, đặc trưng có phân biệt được các truy vấn không, việc huấn luyện có được kiểm soát không, và kết quả đánh giá có phản ánh đúng năng lực của mô hình không. Bảng dưới tóm tắt bằng chứng và khuyến nghị của bài báo cho từng giai đoạn.

*Bảng 1. Tóm tắt bằng chứng thực nghiệm và khuyến nghị của tác giả cho từng giai đoạn*

| **Giai đoạn** | **Bằng chứng trong bài báo** | **Khuyến nghị của tác giả** |
|---|---|---|
| (1) Sinh dữ liệu | Tắt bitmap và tid scan: 28a nhanh hơn 5,5× nhưng 28b cùng mẫu lại chậm hơn 1,9× [§8.4]. Tắt GEQO: 24b chậm hơn 9,9× [§8.5]. | Để PostgreSQL dùng đủ công cụ [§3.2, §8.5]; dùng hot cache, chạy 3 lần và lấy lần thứ 3, vì lần 2 nhanh hơn lần 1 khoảng 14,6% còn lần 3 chỉ nhanh thêm khoảng 1% [§7.3, §8.6]. |

| | | |
|---|---|---|
| (2) Mã hóa | Bao huấn luyện trên IMDB đã giảm 50% dòng: truy vấn 31c chậm hơn 24× [§8.3]. | Dùng embedding [§4.1]; mã hóa cả truy vấn lẫn kế hoạch [§4.2]. |
| (3) Huấn luyện | Chỉ có phân tích, không có thí nghiệm riêng. Balsa (40–85 giờ) và LEON (110–130 giờ) huấn luyện lâu nhưng đứng hai vị trí cuối; Bao chỉ cần khoảng 2 giờ [§8.2.2]. | Dùng tập holdout cố định [§5.1]; dùng learning-to-rank hoặc tách mô hình như HybridQO [§5.2]. |
| 4) Đánh giá | LEON thực thi 58 giây nhưng suy luận khoảng 9,6 giờ [§8.2.1]. | Chia dữ liệu theo LOO / Random / Base Query [§7.2]; đo thời gian đầu-cuối [§8.2.1]; kiểm tra khi dữ liệu thay đổi [§6.2]. |

### 1.3. Kết quả và điều nhóm rút ra

Khi đánh giá trong cùng điều kiện, PostgreSQL nhìn chung vẫn cho kết quả tốt nhất, tiếp theo là HybridQO, Bao, Neo, Balsa và LEON; HybridQO và Bao chỉ vượt hoặc ngang PostgreSQL ở một số split [1, §8.2.1]. Như vậy, các LQO hiện tại chưa tốt hơn PostgreSQL một cách nhất quán. Nhóm rút ra rằng phần lớn vấn đề nằm ở dữ liệu và quy trình đánh giá, không nằm ở độ phức tạp của mô hình. Đây là cơ sở cho hướng phát triển ở Mục 3: dùng mô hình đơn giản nhưng tuân thủ chặt quy trình đánh giá của bài báo.

## 2. Tính SOTA của bài báo

Bài báo [1] không đạt SOTA về hiệu năng vì không đề xuất bộ tối ưu truy vấn mới. Nghiên cứu tập trung cải thiện cách đánh giá các LQO để việc so sánh giữa các phương pháp công bằng hơn.

Trước [1], việc đánh giá các LQO chưa thống nhất. Các phương pháp như Neo [2], Bao [3], Balsa [4], LEON [5], HybridQO [6] và Lero [7] đều báo cáo kết quả tốt hơn PostgreSQL, nhưng sử dụng dữ liệu, cách chia train/test và cấu hình database khác nhau. Một số nghiên cứu đã đề xuất cách chia dữ liệu chung như Ext-JOB [2], JOB-Slow [4] và chia theo số lượng join [8]. Tuy nhiên, các cách này vẫn có hạn chế. Chẳng hạn, số lượng join gần như không phản ánh thời gian thực thi query.

Những đóng góp chính của nghiên cứu [1]:

1. **Xây dựng benchmark chung:** Thống nhất dữ liệu, mã nguồn, cấu hình PostgreSQL và cách chia query để so sánh kết quả.
2. **Thử nghiệm ba cách chia dữ liệu:** Leave One Out, Random và Base Query để kiểm tra khả năng tổng quát hóa của mô hình.

3. **Đo thời gian end-to-end:** Tính cả thời gian suy luận. Ví dụ, LEON [5] mất khoảng 9,6 giờ trên JOB với cách chia Leave One Out.
4. **Đánh giá lại 5 LQO** [2]–[6] trên JOB [9] và STACK [3]. Kết quả cho thấy PostgreSQL vẫn tốt nhất trong phần lớn trường hợp, tiếp theo là HybridQO [6] và Bao [3].
5. **Phân tích ảnh hưởng của cấu hình:** Chỉ ra các lựa chọn như tắt GEQO, bitmap/tid scan hoặc loại bỏ bushy tree có thể làm thay đổi kết quả đánh giá.

Bài báo củng còn hạn chế khi chỉ mới thử nghiệm trên PostgreSQL 12.5 và hai bộ dữ liệu JOB, STACK. Lero [7] không được đánh giá lại do khó tích hợp vào benchmark chung.

# 3. Phát triển từ bài báo (What, How, Why)

## 3.1. WHAT – Bài toán nhóm đề xuất

Lehmann và cộng sự [1, §8.2.1] chỉ ra rằng các phương pháp tối ưu truy vấn ứng dụng học máy (LQO) chưa cải thiện nhất quán tổng thời gian xử lý so với bộ tối ưu truy vấn truyền thống của PostgreSQL. Nguyên nhân đáng chú ý là chi phí suy luận có thể làm mất lợi ích từ kế hoạch thực thi nhanh hơn. Vì vậy, nhóm đề xuất mô hình phân loại có giám sát để hỗ trợ PostgreSQL lựa chọn cấu hình thực thi, thay vì thay thế bộ tối ưu sẵn có.

Với mỗi câu SQL, mô hình dự đoán một trong sáu lựa chọn ở Bảng 2: giữ cấu hình mặc định hoặc hạn chế một loại toán tử JOIN/SCAN. Ý tưởng điều chỉnh bằng hint set tham khảo Bao [1, §2]; sáu lớp dự đoán và mô hình phân loại là đề xuất của nhóm. Các thiết lập off chỉ hạn chế lựa chọn toán tử, không bảo đảm loại bỏ hoàn toàn toán tử đó.

*Bảng 2. Các phương án lựa chọn của mô hình đề xuất*

| Phương án | Ý nghĩa |
|---|---|
| default | Giữ nguyên cách tối ưu mặc định của PostgreSQL. |
| no_nestloop | Hạn chế Nested Loop Join, phương pháp kết hợp dữ liệu bằng vòng lặp lồng nhau. |
| no_hashjoin | Hạn chế Hash Join, phương pháp kết hợp dữ liệu bằng bảng băm. |
| no_mergejoin | Hạn chế Merge Join, phương pháp kết hợp hai nguồn dữ liệu theo thứ tự đã sắp xếp. |
| no_seqscan | Hạn chế Sequential Scan, phương pháp quét tuần tự dữ liệu trong bảng. |
| no_indexscan | Hạn chế Index Scan, phương pháp truy xuất dữ liệu thông qua chỉ mục. |

## 3.2. HOW – Cách thực hiện

Nhóm xây dựng đặc trưng đầu vào (features) từ câu SQL và kế hoạch mặc định do EXPLAIN cung cấp. SQL cho biết số bảng, số phép JOIN và điều kiện lọc; EXPLAIN cung cấp loại JOIN/SCAN, chi phí và số dòng ước lượng mà không cần thực thi truy vấn. Việc kết hợp đặc trưng truy vấn với đặc trưng kế hoạch dựa trên khuyến nghị về Query & Plan Encoding [1, §4.2]. Không sử dụng thời gian chạy thực tế làm đầu vào dự đoán.

Để tạo nhãn (label), mỗi truy vấn huấn luyện được đo với cả sáu cấu hình; phương án nhanh nhất chỉ được chọn thay default khi cải thiện ít nhất 5%. Ngưỡng 5% do nhóm đặt ra nhằm hạn chế nhiễu đo. Dữ liệu dự kiến gồm 113 truy vấn JOB và tập CEB-IMDb bổ sung do nhóm lựa chọn; CEB-IMDb không phải workload đánh giá chính của bài báo gốc [1, §3.1]. Mỗi cấu hình chạy ba lần và lấy lần thứ ba nhằm ổn định trạng thái hot cache [1, §7.3; §8.6].

Nhóm so sánh ba thuật toán phân loại: Decision Tree (mốc cơ sở), Random Forest (mô hình chính) và LightGBM (mô hình cải tiến cần kiểm chứng). Cả ba cùng dự đoán một trong sáu lựa chọn ở Bảng 2; ưu điểm và hạn chế được tóm tắt trong Bảng 3. Đây là lựa chọn nghiên cứu của nhóm, không phải ba mô hình được Lehmann và cộng sự thử nghiệm. Để kiểm tra khả năng tổng quát hóa, nhóm sử dụng Leave One Out, Random Sampling và đặc biệt Base Query Sampling, trong đó các biến thể của cùng một mẫu truy vấn không xuất hiện đồng thời ở tập train và test [1, §7.2].

*Bảng 3. So sánh các mô hình phân loại được nhóm lựa chọn*

| Mô hình | Điểm mạnh | Hạn chế | Vai trò |
|---|---|---|---|
| Decision Tree | Dễ hiểu, dễ giải thích quy tắc chọn hint set, chi phí dự đoán thấp. | Dễ học quá khớp, kết quả nhạy với dữ liệu huấn luyện. | Baseline |
| Random Forest | Kết hợp nhiều cây, thường ổn định hơn cây đơn và xử lý tốt đặc trưng dạng bảng. | Khó diễn giải từng quyết định hơn; vẫn có thể dự đoán sai với truy vấn mới. | Mô hình chính |
| LightGBM | Học được quan hệ đặc trưng phức tạp, suy luận thường nhanh với mô hình nhỏ. | Dễ quá khớp khi ít mẫu, cần điều chỉnh tham số và kiểm tra tổng chi phí. | Mô hình cải tiến để thử nghiệm |

Tiêu chí đánh giá chính là tổng thời gian End-to-End, bao gồm lấy đặc trưng (cả EXPLAIN), suy luận, lập kế hoạch và thực thi [1, §8.2.1]. Nhóm so sánh với bộ tối ưu mặc định của PostgreSQL, Bao và oracle (mốc lý tưởng chọn phương án nhanh nhất sau khi đã đo cả sáu cấu hình). Đồng thời, ghi nhận số truy vấn nhanh hơn hoặc chậm hơn; độ chính xác phân loại chỉ là chỉ số bổ trợ. Chi phí EXPLAIN cần được tính đầy đủ để tránh kết luận sai về lợi ích thực tế.

### 3.3. WHY – Tại sao làm như vậy

Hướng tiếp cận này nhằm kiểm tra liệu mô hình phân loại gọn nhẹ có giảm tổng thời gian truy vấn hay không. Decision Tree dễ diễn giải; Random Forest giảm phụ thuộc vào một cây đơn; LightGBM được bổ sung để kiểm tra khả năng cải thiện dự đoán. Tuy nhiên, dữ liệu hạn chế có thể gây quá khớp, và mô hình nào hiệu quả nhất chỉ được xác định qua thực nghiệm [1, §5.1; §8.2.1].

Việc giữ lựa chọn default là cần thiết vì điều chỉnh toán tử không luôn mang lại lợi ích: bài báo cho thấy cùng một thay đổi có thể giúp truy vấn này nhanh hơn nhưng làm truy vấn khác chậm đi [1, §3.2; §8.4]. Ngưỡng 5% khi gán nhãn chỉ giúp giảm tác động của sai số đo, không loại bỏ rủi ro dự đoán sai.

Để kết luận đáng tin cậy, nhóm đánh giá End-to-End, kiểm thử Base Query Sampling, áp dụng hot cache và thống nhất cấu hình PostgreSQL [1, §7.1; §7.2; §7.3]. Hướng phát triển được xem là có triển vọng nếu giảm tổng thời gian xử lý mà không gây suy giảm lớn trên các truy vấn khác; mọi lợi ích hiện mới là giả thuyết cần kiểm chứng.

# 4. Cài đặt minh họa và kế hoạch thực nghiệm

Nhóm dùng thêm mã nguồn và log mà tác giả công bố tại github.com/edualc/lqo_ml_perspective.

Link github thực nghiệm của nhóm: https://github.com/vipham999/lqo-datamining

## 4.1. Môi trường

*Bảng 4. Môi trường thực nghiệm so với bài báo*

| | **Bài báo nhóm đang tìm hiểu (Lehmann et al., 2024) [1, §8.1]** | **Nhóm** |
|---|---|---|
| **Máy** | x86, 16 CPU, 64 GB RAM, GPU T4 | MacBook M1 Pro, 16 GB RAM (PostgreSQL, Bao); Kaggle, khoảng 30 GB RAM (HybridQO) |
| **DBMS** | PostgreSQL 12.5 + pg_hint_plan | Giống bài báo |
| **shared_buffers / work_mem** | 32 GB / 4 GB | Mac: 4 GB / 256 MB; Kaggle: 8 GB / 512 MB |
| **Dữ liệu** | IMDB, JOB 113 câu, 9 split | Giống bài báo |

## 4.2. Thí nghiệm 1: Kiểm chứng kết luận của bài báo

*Bảng 5. Thiết kế thí nghiệm 1*

| | **Mô tả** |
|---|---|
| **Câu hỏi nghiên cứu** | Kết luận của bài báo có còn đúng trên máy của nhóm không? |
| **Chạy lại** | PostgreSQL; Bao (9 split, Mac); HybridQO (9 split, Kaggle, 15 epoch) |
| **Dùng log tác giả** | Neo, Balsa, LEON (cần GPU, 20 - 130 giờ mỗi split) |
| **Ablation** | Tắt GEQO, tắt bitmap/tid scan [1, §8.4 - 8.5] |
| **Kết quả trình bày** | Thắng/hòa/thua (±10%) và speedup so với PostgreSQL theo từng split, cạnh số của bài báo |

## 4.3. Thí nghiệm 2: Đánh giá hướng phát triển của nhóm

*Bảng 6. Thiết kế thí nghiệm 2*

| | **Mô tả** |
|---|---|
| **Câu hỏi nghiên cứu** | Mô hình phân loại đơn giản có giúp PostgreSQL nhanh hơn không? |
| **Dữ liệu** | 113 câu JOB và khoảng 200 câu CEB-IMDb |
| **Nhãn** | Phương án nhanh nhất; giữ default nếu chênh dưới 5% |
| **Đặc trưng** | Câu SQL và kế hoạch EXPLAIN (có trước khi chạy) |
| **Mô hình** | Decision Tree, Random Forest, LightGBM |
| **Mốc so sánh** | PostgreSQL mặc định; Bao; Oracle (biết trước phương án nhanh nhất của từng câu, là kết quả tốt nhất có thể đạt) |
| **Cách chia dữ liệu** | Leave One Out, Random, Base Query [1, §7.2] |
| **Kết quả trình bày** | Tổng thời gian chạy; số câu nhanh lên, chậm đi; luật chọn phương án |
| **Giả thuyết** | H1: mô hình nhanh hơn PostgreSQL và không thua Bao. H2: mô hình kém nhất với loại câu chưa từng học |

## 4.4. Hạn chế

Bộ nhớ thấp hơn bài báo nên nhóm chỉ so sánh xu hướng và thứ hạng, không so sánh số tuyệt đối. HybridQO huấn luyện 15 thay vì 50 epoch, vì log của tác giả cho thấy kết quả không đổi từ epoch 10. Oracle chỉ là phương án nhanh nhất trong 6 phương án, chưa phải kế hoạch tối ưu toàn cục.

# Tài liệu tham khảo


[1] C. Lehmann, P. Sulimov, K. Stockinger, "Is Your Learned Query Optimizer Behaving As You Expect? A Machine Learning Perspective," PVLDB, vol. 17, no. 7, pp. 1565–1577, 2024.

[2] R. Marcus et al., "Neo: A Learned Query Optimizer," PVLDB, vol. 12, no. 11, 2019.

[3] R. Marcus et al., "Bao: Making Learned Query Optimization Practical," in Proc. SIGMOD, 2021, pp. 1275–1288.

[4] Z. Yang et al., "Balsa: Learning a Query Optimizer Without Expert Demonstrations," in Proc. SIGMOD, 2022, pp. 931–944.

[5] X. Chen et al., "LEON: A New Framework for ML-Aided Query Optimization," PVLDB, vol. 16, no. 9, pp. 2261–2273, 2023.

[6] X. Yu et al., "Cost-based or Learning-based? A Hybrid Query Optimizer for Query Plan Selection," PVLDB, vol. 15, no. 13, pp. 3924–3936, 2022.

[7] R. Zhu et al., "Lero: A Learning-to-Rank Query Optimizer," arXiv:2302.06873, 2023.

[8] Y. Zhang et al., "Simple Adaptive Query Processing vs. Learned Query Optimizers: Observations and Analysis," PVLDB, vol. 16, no. 9, pp. 2962–2975, 2023.

[9] V. Leis et al., "How Good Are Query Optimizers, Really?," PVLDB, vol. 9, no. 3, pp. 204–215, 2015.